

LAPORAN PRAKTIKUM

STRUKTUR DATA

Dosen Pengampu : Dr. Wahyudi, S.T, M.T



Disusun Oleh:
Annafi Al Ghifari
2511533013

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan tugas mata kuliah *Struktur Data* ini dengan baik dan tepat waktu.

Laporan/tugas ini disusun sebagai salah satu bentuk pemenuhan kewajiban akademik pada mata kuliah Struktur Data. Dalam penyusunannya, penulis berusaha memahami konsep-konsep dasar yang berkaitan dengan struktur data, seperti pengelolaan data menggunakan array, stack, queue, serta implementasi dalam pemrograman. Diharapkan melalui tugas ini, penulis dapat meningkatkan pemahaman serta kemampuan dalam menerapkan konsep struktur data secara praktis.

Penulis menyadari bahwa dalam penyusunan tugas ini masih terdapat banyak kekurangan, baik dari segi isi maupun penyajian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Struktur Data serta semua pihak yang telah membantu dalam proses penyusunan tugas ini.

Semoga laporan ini dapat memberikan manfaat bagi penulis khususnya dan bagi pembaca pada umumnya.

Padang, 28 April 2026

Annafi Al Ghifari

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
BAB II PEMBAHASAN	2
2.1 Program Dan Kodingan.....	2
2.2 Tugas.....	10
BAB III KESIMPULAN.....	17
3.1 Ringkasan.....	17
3.2 Saran	17
DAFTAR PUSTAKA	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi menuntut mahasiswa untuk memahami konsep dasar dalam pemrograman, salah satunya adalah struktur data. Struktur data merupakan cara untuk mengorganisasi dan mengelola data agar dapat digunakan secara efisien. Salah satu struktur data yang penting untuk dipahami adalah Queue (antrian), yang bekerja berdasarkan prinsip FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar.

Dalam kehidupan sehari-hari, konsep antrian banyak dijumpai, seperti pada antrian loket pelayanan, kasir, maupun sistem pelayanan publik lainnya. Oleh karena itu, pemahaman mengenai Queue menjadi sangat relevan karena dapat diaplikasikan dalam berbagai sistem nyata. Implementasi Queue dalam pemrograman memungkinkan mahasiswa untuk memahami bagaimana data dikelola secara berurutan dan sistematis.

Pada tugas ini, Queue diimplementasikan menggunakan array dalam bahasa pemrograman Java. Program yang dibuat mensimulasikan antrian pelanggan pada loket pelayanan, di mana setiap pelanggan akan dilayani berdasarkan urutan kedatangan. Selain itu, program juga dilengkapi dengan berbagai operasi dasar seperti penambahan data (enqueue), penghapusan data (dequeue), pengecekan kondisi antrian, serta fitur tambahan seperti reverse untuk membalik urutan antrian.

Dengan mengerjakan tugas ini, diharapkan mahasiswa dapat memahami konsep dasar Queue, cara implementasinya dalam bahasa pemrograman, serta mampu mengembangkan logika pemrograman yang lebih terstruktur dan sistematis.

BAB II

PEMBAHASAN

2.1 Program Dan Kodingan

1. HapusDLL_2511533013

```
2. package pekan6_2511533013;
3.
4. public class HapusDLL_2511533013 {
5.     // fungsi menghapus node awal
6.     public static NodeDLL_2511533013
delHead_3013(NodeDLL_2511533013 head_3013) {
7.         if (head_3013 == null) {
8.             return null;
9.         }
10.        NodeDLL_2511533013 temp = head_3013;
11.        head_3013 = head_3013.next;
12.        if (head_3013 != null) {
13.            head_3013.prev = null; }
14.        return head_3013;
15.    }
16.    // fungsi menghapus di akhir
17.    public static NodeDLL_2511533013
delEnd_3013(NodeDLL_2511533013 head_3013) {
18.        if (head_3013 == null) {
19.            return null;
20.        }
21.        if (head_3013.next == null) {
22.            return null;
23.        }
24.        NodeDLL_2511533013 curr = head_3013;
25.        while (curr.next != null) {
26.            curr = curr.next;
27.        }
28.        // update pointer previous node
29.        if (curr.prev != null) {
30.            curr.prev.next = null; }
31.        return head_3013;
32.    }
33.    }
34.    // fungsi menghapus node di posisi tertentu
```

```

35. public static NodeDLL_2511533013 delPos_3013(NodeDLL_2511533013
    head_3013, int pos_3013) {
36.
37.     // jika DLL kosong
38.     if (head_3013 == null) {
39.         return head_3013;
40.     }
41.
42.     NodeDLL_2511533013 curr = head_3013;
43.
44.     // telusuri sampai node yang akan dihapus
45.     for (int i = 1; curr != null && i < pos_3013; ++i) {
46.         curr = curr.next;
47.     }
48.
49.     // jika posisi tidak ditemukan
50.     if (curr == null) {
51.         return head_3013;
52.     }
53.
54.     // update pointer prev dan next
55.     if (curr.prev != null) {
56.         curr.prev.next = curr.next;
57.     }
58.
59.     if (curr.next != null) {
60.         curr.next.prev = curr.prev;
61.     }
62.
63.     // jika node yang dihapus adalah head
64.     if (head_3013 == curr) {
65.         head_3013 = curr.next;
66.     }
67.
68.     return head_3013;
69. }
70.
71. // fungsi mencetak DLL
72. public static void printList(NodeDLL_2511533013 head_3013) {
73.
74.     NodeDLL_2511533013 curr = head_3013;
75.
76.     while (curr != null) {

```

```

77.         System.out.print(curr.data + " ");
78.         curr = curr.next;
79.     }
80.
81.     System.out.println();
82. }
83.     public static void main(String[] args) {
84.         // buat sebuah DLL
85.         NodeDLL_2511533013 head_3013 = new
NodeDLL_2511533013(1);
86.         head_3013.next = new NodeDLL_2511533013(2);
87.         head_3013.next.prev = head_3013;
88.         head_3013.next.next = new NodeDLL_2511533013(3);
89.         head_3013.next.next.prev = head_3013.next;
90.         head_3013.next.next.next = new
NodeDLL_2511533013(4);
91.         head_3013.next.next.next.prev =
head_3013.next.next;
92.         head_3013.next.next.next.next = new
NodeDLL_2511533013(5);
93.         head_3013.next.next.next.next.prev =
head_3013.next.next.next;
94.
95.         System.out.println("DLL awal:");
96.         printList(head_3013);
97.
98.         System.out.println("Setelah head dihapus:");
99.         head_3013 = delHead_3013(head_3013);
100.            printList(head_3013);
101.
102.            System.out.println("Setelah node terakhir
dihapus:");
103.            head_3013 = delEnd_3013(head_3013);
104.            printList(head_3013);
105.
106.            System.out.println("menghapus node ke 2:");
107.            head_3013 = delPos_3013(head_3013, 2);
108.            printList(head_3013);
109.        }
110.    }
111.
112.
113.

```

OUTPUT:

```

DLL awal:
1 2 3 4 5
Setelah head dihapus:
2 3 4 5
Setelah node terakhir dihapus:
2 3 4
menghapus node ke 2:
2 4
PS C:\Users\annaf\git\repository\PrakSD2026_C_2511533013\src>

```

2. InsertDLL_2511533013

```

3. package pekan6_2511533013;
4.
5. public class InsertDLL_2511533013 {
6.     // menambahkan node di awal DLL
7.     static NodeDLL_2511533013 insertBegin(NodeDLL_2511533013
head_3013, int data_3013) {
8.         // buat node baru
9.         NodeDLL_2511533013 new_node = new
NodeDLL_2511533013(data_3013);
10.        // jadikan pointer next head
11.        new_node.next = head_3013;
12.        // jadikan pointer prev head ke new_node
13.        if (head_3013 != null) {
14.            head_3013.prev = new_node;
15.        }
16.        // kembalikan pointer ke node baru
17.        return new_node;
18.    }
19.    // fungsi mmenambahkan node di akhir
20.    public static NodeDLL_2511533013
insertEnd(NodeDLL_2511533013 head_3013, int newData_3013) {
21.        // buat node baru
22.        NodeDLL_2511533013 new_node = new
NodeDLL_2511533013(newData_3013);
23.        // jika dll null dijadikan head
24.        if(head_3013 == null) {
25.            head_3013 = new_node;

```



```

26.     }
27.     else {
28.         NodeDLL_2511533013 curr = head_3013;
29.         while (curr.next != null) {
30.             curr = curr.next;
31.         }
32.         curr.next = new_node;
33.         new_node.prev = curr;
34.     }
35.     return head_3013;
36. }
37. // fungsi menambahkan node di posisi tertentu
38. public static NodeDLL_2511533013
    insertAtPosition_3013(NodeDLL_2511533013 head_3013, int
    position_3013, int new_data_3013) {
39.     // buat node baru
40.     NodeDLL_2511533013 new_node = new
    NodeDLL_2511533013(new_data_3013);
41.     if (position_3013 == 1) {
42.         new_node.next = head_3013;
43.         if (head_3013 != null) {
44.             head_3013.prev = new_node; }
45.         head_3013 = new_node;
46.         return head_3013; }
47.     NodeDLL_2511533013 curr = head_3013;
48.     for (int i = 1; i < position_3013 - 1 && curr
    != null; ++i) {
49.         curr = curr.next;
50.         if (curr == null) {
51.             System.out.println("Posisi tidak ada");
52.             return head_3013; }
53.         new_node.prev = curr;
54.         new_node.next = curr.next;
55.         curr.next = new_node;
56.         if (new_node.next != null) {
57.             new_node.next.prev = new_node; }
58.         return head_3013; }
59.     return curr;
60. }
61. public static void printList(NodeDLL_2511533013 head_3013)
    {
62.     NodeDLL_2511533013 curr = head_3013;
63.     while (curr != null) {

```

```

64.         System.out.print(curr.data + " <-> ");
65.         curr = curr.next;
66.     }
67.     System.out.println();
68. }
69. public static void main(String[] args) {
70.     // membuat dll 2 <-> 3 <-> 5
71.     NodeDLL_2511533013 head_3013 = new
NodeDLL_2511533013(2);
72.     head_3013.next = new NodeDLL_2511533013(3);
73.     head_3013.next.prev = head_3013;
74.     head_3013.next.next = new NodeDLL_2511533013(5);
75.     head_3013.next.next.prev = head_3013.next;
76.     // cetak dll awal
77.     System.out.println("DLL awal:");
78.     printList(head_3013);
79.     // tambah 1 di awal
80.     head_3013 = insertBegin(head_3013, 1);
81.     System.out.println("Setelah menambahkan 1 di awal:");
82.     printList(head_3013);
83.     // tambah 6 di akhir
84.     System.out.println("Simpul 6 ditambah di akhir:");
85.     head_3013 = insertEnd(head_3013, 6);
86.     printList(head_3013);
87.     // menambah node 4 di posisi 4
88.     System.out.println("tambah node 4 di posisi 4:");
89.     head_3013 = insertAtPosition_3013(head_3013, 4, 4);
90.     printList(head_3013);
91. }
92. }
93.

```

OUTPUT:

```

PS C:\Users\annaf\git\repository\PrakSD2026_C_2511533013\src> & 'C:\Program Fi
'-cp' 'C:\Users\annaf\AppData\Roaming\Code\User\workspaceStorage\4d61957f9bbdd
3013.InsertDLL_2511533013'
DLL awal:
2 <-> 3 <-> 5 <->
Setelah menambahkan 1 di awal:
1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir:
1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4:
1 <-> 2 <-> 4 <-> 3 <-> 5 <-> 6 <->
PS C:\Users\annaf\git\repository\PrakSD2026_C_2511533013\src>

```

```

aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa

```

3. NodeDLL_2511533013

```
4. package pekan6_2511533013;
5.
6. public class NodeDLL_2511533013 {
7.     // mendefinisikan kelas node
8.     int data; //data
9.     NodeDLL_2511533013 next; //pointer ke node berikutnya
10.    NodeDLL_2511533013 prev; //pointer ke node sebelumnya
11.
12.    // konstruktor
13.    public NodeDLL_2511533013(int data) {
14.        this.data = data;
15.        this.next = null;
16.        this.prev = null;
17.    }
18.}
19.
```

4. PenelusuranDLL_2511533013

```
5. package pekan6_2511533013;
6.
7. public class PenelusuranDLL_2511533013 {
8.     // fungsi penelusuran maju
9.     static void forwardTraversal(NodeDLL_2511533013 head_3013)
10.    {
11.        // memulai penelusuran dari head
12.        NodeDLL_2511533013 curr = head_3013;
13.        // lanjutkan sampai akhir
14.        while (curr != null) {
15.            // print data
16.            System.out.println(curr.data + " <->");
17.            // pindah ke node berikutnya
18.            curr = curr.next;
19.        }
20.        // print spasi
21.        System.out.println();
22.    }
23.    // fungsi penelusuran mundur
24.}
```

```

23.         static void backwardTraversal(NodeDLL_2511533013
tail_3013) {
24.             // mulai dari akhir
25.             NodeDLL_2511533013 curr = tail_3013;
26.             // lanjutkan sampai head
27.             while (curr != null) {
28.                 // print data
29.                 System.out.println(curr.data + " <->");
30.                 // pindah ke node sebelumnya
31.                 curr = curr.prev;
32.             }
33.             // print spasi
34.             System.out.println();
35.         }
36.         public static void main(String[] args) {
37.             // cetak DLL
38.             NodeDLL_2511533013 head_3013 = new
NodeDLL_2511533013(1);
39.             NodeDLL_2511533013 second_3013 = new
NodeDLL_2511533013(2);
40.             NodeDLL_2511533013 third_3013 = new
NodeDLL_2511533013(3);
41.
42.             head_3013.next = second_3013;
43.             second_3013.prev = head_3013;
44.             second_3013.next = third_3013;
45.             third_3013.prev = second_3013;
46.
47.             System.out.println("Penelusuran maju:");
48.             forwardTraversal(head_3013);
49.
50.             System.out.println("Penelusuran mundur:");
51.             backwardTraversal(third_3013);
52.         }
53.     }
54.
55.

```

OUTPUT:

<pre>Penelusuran maju: 1 <-> 2 <-> 3 <-> Penelusuran mundur: 3 <-> 2 <-> 1 <-> PS C:\Users\annaf\git\repository\PrakSD2026_C_2511533013\src></pre>	
---	--

2.2 Tugas

1. Lagu_2511533013

```
4. package pekan6_2511533013;
5.
6. public class NodeDLL_2511533013 {
7.     // mendefinisikan kelas node
8.     int data; //data
9.     NodeDLL_2511533013 next; //pointer ke node berikutnya
10.    NodeDLL_2511533013 prev; //po
11.
```

2. Musik_2511533013

```
1. package pekan6_2511533013;
2.
3. public class Musik_2511533013 {
4.     Lagu_2511533013 head_3013;
5.     Lagu_2511533013 tail_3013;
6.
7.     // method menambah lagu di akhir playlist
8.     public void tambahLagu_3013(String judul_3013, String
penyanyi_3013) {
9.         Lagu_2511533013 laguBaru_3013 = new
Lagu_2511533013(judul_3013, penyanyi_3013);
10.
11.         // jika playlist kosong
12.         if (head_3013 == null) {
13.             head_3013 = laguBaru_3013;
14.             tail_3013 = laguBaru_3013;
15.         } else {
16.             tail_3013.next_3013 = laguBaru_3013;
17.             laguBaru_3013.prev_3013 = tail_3013;
18.             tail_3013 = laguBaru_3013;
19.         }
}
```

```

20.
21.         System.out.println("Lagu berhasil ditambahkan!\n");
22.     }
23.
24.     // method menghapus lagu pertama
25.     public void hapusLaguAwal_3013() {
26.         // jika playlist kosong
27.         if (head_3013 == null) {
28.             System.out.println("Playlist kosong!\n");
29.             return;
30.         }
31.
32.         // jika hanya ada satu lagu
33.         if (head_3013 == tail_3013) {
34.             System.out.println("Lagu " +
head_3013.getJudul_3013() + " berhasil dihapus!\n");
35.             head_3013 = null;
36.             tail_3013 = null;
37.         } else {
38.             System.out.println("Lagu " +
head_3013.getJudul_3013() + " berhasil dihapus!\n");
39.             head_3013 = head_3013.next_3013;
40.             head_3013.prev_3013 = null;
41.         }
42.     }
43.
44.     // method menampilkan playlist dari awal ke akhir
45.     public void tampilMaju_3013() {
46.         // jika playlist kosong
47.         if (head_3013 == null) {
48.             System.out.println("Playlist kosong!\n");
49.             return;
50.         }
51.
52.         Lagu_2511533013 bantu_3013 = head_3013;
53.
54.         System.out.println("=== Playlist Maju ===");
55.         while (bantu_3013 != null) {
56.             System.out.println("Judul      : " +
bantu_3013.getJudul_3013());
57.             System.out.println("Penyanyi  : " +
bantu_3013.getPenyanyi_3013());
58.             System.out.println();
59.
60.             bantu_3013 = bantu_3013.next_3013;
61.         }
62.     }
63.
64.     // method menampilkan playlist dari akhir ke awal
65.     public void tampilMundur_3013() {
66.         // jika playlist kosong
67.         if (tail_3013 == null) {
68.             System.out.println("Playlist kosong!\n");

```

```

69.         return;
70.     }
71.
72.     Lagu_2511533013 bantu_3013 = tail_3013;
73.
74.     System.out.println("=== Playlist Mundur ===");
75.     while (bantu_3013 != null) {
76.         System.out.println("Judul      : " +
bantu_3013.getJudul_3013());
77.         System.out.println("Penyanyi  : " +
bantu_3013.getPenyanyi_3013());
78.         System.out.println();
79.
80.         bantu_3013 = bantu_3013.prev_3013;
81.     }
82. }
83.
84. // method mencari lagu berdasarkan judul
85. public void cariLagu_3013(String judulCari_3013) {
86.     // jika playlist kosong
87.     if (head_3013 == null) {
88.         System.out.println("Playlist kosong!\n");
89.         return;
90.     }
91.
92.     Lagu_2511533013 bantu_3013 = head_3013;
93.     boolean ditemukan_3013 = false;
94.
95.     while (bantu_3013 != null) {
96.         if
(bantu_3013.getJudul_3013().equalsIgnoreCase(judulCari_3013)) {
97.             System.out.println("Lagu ditemukan!");
98.             System.out.println("Judul      : " +
bantu_3013.getJudul_3013());
99.             System.out.println("Penyanyi  : " +
bantu_3013.getPenyanyi_3013());
100.            System.out.println();
101.            ditemukan_3013 = true;
102.            break;
103.        }
104.
105.        bantu_3013 = bantu_3013.next_3013;
106.    }
107.
108.    if (!ditemukan_3013) {
109.        System.out.println("Lagu tidak ditemukan!\n");
110.    }
111. }
112. }
113.
114.

```

3. MainPlaylist_2511533013

```
1. package pekan6_2511533013;
2.
3. import java.util.Scanner;
4.
5. public class MainPlaylist_2511533013 {
6.     public static void main(String[] args) {
7.         Scanner input_3013 = new Scanner(System.in);
8.         Musik_2511533013 playlist_3013 = new
Musik_2511533013();
9.
10.        int pilihan_3013;
11.
12.        do {
13.            System.out.println("=== Playlist Musik ===");
14.            System.out.println("1. Tambah Lagu");
15.            System.out.println("2. Hapus Lagu Pertama");
16.            System.out.println("3. Lihat Playlist (Maju)");
17.            System.out.println("4. Lihat Playlist
(Mundur)");
18.            System.out.println("5. Cari Lagu");
19.            System.out.println("6. Keluar");
20.            System.out.print("Pilihan : ");
21.            pilihan_3013 = input_3013.nextInt();
22.            input_3013.nextLine();
23.
24.            switch (pilihan_3013) {
25.                case 1:
26.                    System.out.print("Judul Lagu : ");
27.                    String judul_3013 =
input_3013.nextLine();
28.
29.                    System.out.print("Penyanyi   : ");
30.                    String penyanyi_3013 =
input_3013.nextLine();
31.
32.                    playlist_3013.tambahLagu_3013(judul_3013
, penyanyi_3013);
33.                    break;
34.
35.                case 2:
36.                    playlist_3013.hapusLaguAwal_3013();
37.                    break;
```



```

38.
39.         case 3:
40.             playlist_3013.tampilMaju_3013();
41.             break;
42.
43.         case 4:
44.             playlist_3013.tampilMundur_3013();
45.             break;
46.
47.         case 5:
48.             System.out.print("Masukkan judul lagu
yang dicari : ");
49.             String cari_3013 =
input_3013.nextLine();
50.
51.             playlist_3013.cariLagu_3013(cari_3013);
52.             break;
53.
54.         case 6:
55.             System.out.println("Program selesai.");
56.             break;
57.
58.         default:
59.             System.out.println("Pilihan tidak
valid!\n");
60.     }
61.
62.     } while (pilihan_3013 != 6);
63.
64.     input_3013.close();
65. }
66. }

```

OUTPUT:

<pre> === Playlist Musik === 1. Tambah Lagu 2. Hapus Lagu Pertama 3. Lihat Playlist (Maju) 4. Lihat Playlist (Mundur) 5. Cari Lagu 6. Keluar Pilihan : 1 Judul Lagu : Chocolate Penyanyi : The 1975 Lagu berhasil ditambahkan! </pre>	<pre> Kita tambahkan judul lagu 1. Gatau 2. Dan 3. Chocolate 4. God's Plan </pre>
---	---

<pre> === Playlist Musik === 1. Tambah Lagu 2. Hapus Lagu Pertama 3. Lihat Playlist (Maju) 4. Lihat Playlist (Mundur) 5. Cari Lagu 6. Keluar Pilihan : 5 Masukkan judul lagu yang dicari : dan Lagu ditemukan! Judul : dan Penyanyi : So7 </pre>	Kita coba cari lagu “dan”, disini kita berhasil menemukannya
<pre> === Playlist Musik === 1. Tambah Lagu 2. Hapus Lagu Pertama 3. Lihat Playlist (Maju) 4. Lihat Playlist (Mundur) 5. Cari Lagu 6. Keluar Pilihan : 3 === Playlist Maju === Judul : gatau Penyanyi : unknown Judul : dan Penyanyi : So7 Judul : Chocolate Penyanyi : The 1975 Judul : God's Plan Penyanyi : Drake </pre>	Disini kita lihat playlist (maju), dan disini kita bisa lihat ada 4 judul lagu yang ter daftar sesuai dengan yang kita input Kalau kita Maju kan lagi, disini kita bisa lihat lagu “gatau” ter skip

<pre> === Playlist Musik === 1. Tambah Lagu 2. Hapus Lagu Pertama 3. Lihat Playlist (Maju) 4. Lihat Playlist (Mundur) 5. Cari Lagu 6. Keluar Pilihan : 3 === Playlist Maju === Judul : dan Penyanyi : So7 Judul : Chocolate Penyanyi : The 1975 Judul : God's Plan Penyanyi : Drake </pre>		
<pre> === Playlist Musik === 1. Tambah Lagu 2. Hapus Lagu Pertama 3. Lihat Playlist (Maju) 4. Lihat Playlist (Mundur) 5. Cari Lagu 6. Keluar Pilihan : 4 === Playlist Mundur === Judul : God's Plan Penyanyi : Drake Judul : Chocolate Penyanyi : The 1975 Judul : dan Penyanyi : So7 Judul : gatau Penyanyi : unknown </pre>		Disini kita lihat playlist (mundur), dan disini kita bisa lihat ada 4 judul lagu, disini lagu “God’s Plan” bisa di atas karena kita kita memundurkan playlist yang posisi seharusnya: 1. gatau 2. dan 3. chocolate 4. god’s plan
<pre> === Playlist Musik === 1. Tambah Lagu 2. Hapus Lagu Pertama 3. Lihat Playlist (Maju) 4. Lihat Playlist (Mundur) 5. Cari Lagu 6. Keluar Pilihan : 2 Lagu gatau berhasil dihapus! </pre>		Disini kita coba hapus lagu pertama, dan kita dapat hasil untuk lagu “gatau” berhasil di hapus

BAB III

KESIMPULAN

3.1 Ringkasan

Berdasarkan praktikum yang telah dilakukan, implementasi Double Linked List dan Queue menggunakan bahasa pemrograman Java berhasil dijalankan dengan baik. Double Linked List mampu melakukan operasi penambahan, penghapusan, dan penelusuran data secara fleksibel karena memiliki pointer next dan prev.

Selain itu, Queue berhasil diimplementasikan menggunakan prinsip FIFO (First In First Out). Operasi enqueue, dequeue, display, isEmpty, isFull, dan reverse dapat berjalan sesuai dengan yang diharapkan.

Melalui praktikum ini, pemahaman mengenai pengelolaan data menggunakan struktur data menjadi lebih baik. Praktikum juga membantu meningkatkan kemampuan dalam menyusun logika pemrograman secara sistematis dan terstruktur.

3.2 Saran

Bagi pengajar, diharapkan dapat memberikan lebih banyak contoh kasus yang relevan dengan kehidupan nyata sehingga mahasiswa dapat lebih mudah memahami penerapan struktur data dalam dunia kerja.

DAFTAR PUSTAKA

1. Kadir, Abdul. 2018. Dasar Pemrograman Java. Yogyakarta: Andi Offset.
2. Nugroho, Adi. 2019. Struktur Data dengan Java. Bandung: Informatika.
3. Sedgewick, Robert dan Kevin Wayne. 2011. Algorithms Fourth Edition. Boston: Addison-Wesley.
4. Oracle Corporation. 2026. Java Documentation. <https://docs.oracle.com/javase/>
5. Weiss, Mark Allen. 2014. Data Structures and Algorithm Analysis in Java. United States: Pearson Education.